

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Materia: Programación 1

TRABAJO PRÁCTICO INTEGRADOR (TPI)

Gestión de Datos de Países en Python:

filtros, ordenamientos y estadísticas

Estudiante: Rodolfo Gaspar Paredes

Cohorte: Marzo 2026

Fecha de entrega: Junio 2026

Índice

1. Introducción	3
2. Marco Teórico	3
2.1 Listas	3
2.2 Diccionarios	4
2.3 Funciones	4
2.4 Condicionales	4
2.5 Ordenamientos	5
2.6 Estadísticas básicas	5
2.7 Archivos CSV	5
3. Decisiones Técnicas y Arquitectura	6
4. Dificultades y Conclusiones	7
5. Bibliografía / Webgrafía	7
6. Links	8

1. Introducción

El presente Trabajo Práctico Integrador (TPI) consiste en el desarrollo de una aplicación en Python que permite gestionar información sobre países del mundo. El sistema implementa operaciones de carga y persistencia de datos desde archivos CSV, búsqueda, filtrado, ordenamiento y generación de estadísticas.

El objetivo principal es afianzar el uso de estructuras de datos fundamentales como listas y diccionarios, la modularización del código mediante funciones con responsabilidad única, y la aplicación de técnicas de filtrado y ordenamiento, integrando los conceptos trabajados a lo largo de la materia Programación 1.

2. Marco Teórico

2.1 Listas

Una lista en Python es una estructura de datos ordenada y mutable que permite almacenar colecciones de elementos del mismo o distintos tipos. Se definen con corchetes y admiten operaciones como agregar, eliminar y acceder a elementos por índice.

En este proyecto, la lista es la estructura principal de almacenamiento: cada elemento de la lista es un diccionario que representa un país. Ejemplo de declaración:

```
países = []  
países.append({'nombre': 'Argentina', 'poblacion': 45376763, ...})
```

Las listas permiten iterar sobre sus elementos con bucles for, lo que facilita la búsqueda y el filtrado de datos.

2.2 Diccionarios

Un diccionario en Python es una estructura de datos que almacena pares clave-valor. A diferencia de las listas, el acceso a los elementos se realiza mediante claves en lugar de índices numéricos. Son mutables y permiten modificar, agregar y eliminar entradas.

En el sistema, cada país se representa como un diccionario con cuatro claves: nombre, poblacion, superficie y continente. Esto permite acceder a cualquier dato del país de forma clara y legible:

```
pais['nombre']      # Retorna el nombre del país  
pais['poblacion']   # Retorna la población como entero
```

2.3 Funciones

Las funciones son bloques de código reutilizables que encapsulan una lógica específica y pueden recibir parámetros y retornar valores. En Python se definen con la palabra clave def.

Este proyecto aplica el principio de responsabilidad única: cada función realiza una sola tarea. Por ejemplo, cargar_paises() solo lee el CSV y retorna la lista, guardar_paises() solo escribe, filtrar_paises() gestiona el submenú de filtros, y así sucesivamente. Esto facilita la lectura, el mantenimiento y la detección de errores.

También se utilizan type hints para documentar el tipo de los parámetros y el valor de retorno:

```
def cargar_paises(archivo: str) -> list:
```

2.4 Condicionales

Las estructuras condicionales permiten ejecutar bloques de código según si una condición es verdadera o falsa. Python utiliza `if`, `elif` y `else` para este fin.

En el sistema se usan ampliamente para validar entradas del usuario, verificar si un archivo existe, controlar el flujo del menú principal y manejar casos sin resultados en búsquedas y filtros:

```
if not os.path.exists(archivo):
    print('[INFO] El archivo no existe.')
    return paises
```

2.5 Ordenamientos

El ordenamiento consiste en reorganizar una colección de elementos según un criterio determinado. Python provee la función integrada `sorted()`, que retorna una nueva lista ordenada sin modificar la original.

El parámetro `key` permite especificar el criterio de ordenamiento mediante una función. El parámetro `reverse=True` invierte el orden (descendente). En este proyecto se utiliza una función anidada para acceder al campo por el que se desea ordenar:

```
def obtener_valor(p):
    return p[campo]

ordenados = sorted(paises, key=obtener_valor, reverse=descendente)
```

Python también permite definir funciones anónimas de una sola expresión mediante la palabra clave `lambda`. Son equivalentes a las arrow functions de JavaScript y resultan útiles cuando la función es simple y de uso puntual. En este proyecto se utilizaron lambdas en las funciones de estadísticas para indicar el criterio de comparación de `max()` y `min()`:

```
max_pob = max(paises, key=lambda p: p['poblacion'])
```

2.6 Estadísticas básicas

Las estadísticas básicas permiten obtener indicadores clave a partir de un conjunto de datos. Python provee funciones integradas como `max()`, `min()` y `sum()` que simplifican estos cálculos.

En el sistema se calculan el país con mayor y menor población y superficie, el promedio de ambas variables, y la cantidad de países por continente. Para indicar el criterio de comparación se utilizaron funciones `lambda`, que son funciones anónimas de una sola expresión. La siguiente línea retorna el país con mayor población:

```
max_pob = max(paises, key=lambda p: p['poblacion'])
```

La expresión `lambda p: p['poblacion']` es equivalente a definir una función que recibe un país `p` y retorna su valor de población.

```
prom_pob = sum(p['poblacion'] for p in paises) / len(paises)
```

Para contar países por continente se utiliza un diccionario acumulador con el método `get()`, que retorna un valor por defecto si la clave no existe:

```
continentes[cont] = continentes.get(cont, 0) + 1
```

2.7 Archivos CSV

CSV (Comma-Separated Values) es un formato de archivo de texto plano donde los datos se organizan en filas y columnas separadas por comas. Es ampliamente utilizado para el intercambio de datos entre aplicaciones por su simplicidad y compatibilidad.

Python incluye el módulo `csv` en su librería estándar, que provee las clases `DictReader` y `DictWriter` para leer y escribir archivos CSV como diccionarios. El módulo `os` complementa esta funcionalidad permitiendo verificar si el archivo existe antes de intentar abrirlo:

```
import csv
import os

with open(archivo, newline='', encoding='utf-8') as f:
    lector = csv.DictReader(f)
    for fila in lector:
        paises.append({...})
```

El parámetro `newline=""` es recomendado por la documentación oficial de Python para evitar problemas con los saltos de línea en sistemas operativos Windows.

3. Decisiones Técnicas y Arquitectura

Estructura del proyecto

El proyecto está compuesto por dos archivos principales:

- `países.py`: código fuente principal con toda la lógica del sistema.
- `países.csv`: dataset con información de 44 países de todos los continentes.

Decisiones de diseño

- Modularización: cada funcionalidad se implementó en una función independiente con una única responsabilidad, siguiendo las consignas del trabajo.
- Persistencia automática: el archivo CSV se actualiza al salir del programa, garantizando que los cambios realizados durante la sesión queden guardados.
- Validaciones de entrada: las funciones `pedir_texto()` y `pedir_entero()` centralizan la validación de entradas, evitando duplicar lógica en cada funcionalidad.
- Manejo de errores: se utilizan bloques `try/except` para manejar errores de lectura/escritura del CSV y conversión de tipos de datos.
- Cobertura geográfica: el dataset incluye países de América, Europa, Asia, África y Oceanía para que los filtros por continente tengan resultados representativos.

Flujo de operaciones principales

El programa sigue el siguiente flujo general al ejecutarse:

1. Inicio: se intenta cargar el archivo `países.csv` en memoria como lista de diccionarios.
2. Menú principal: se presenta un menú con 8 opciones (agregar, actualizar, buscar, filtrar, ordenar, estadísticas, listar, salir).
3. Procesamiento: según la opción elegida, se llama a la función correspondiente que opera sobre la lista en memoria.
4. Cierre: al seleccionar la opción 0, se guarda la lista actualizada en el CSV y el programa termina.

Capturas de pantalla

Ver el video demostrativo y el repositorio de GitHub para ejemplos de ejecución. El `README.md` del repositorio incluye capturas de las principales funcionalidades.

4. Dificultades y Conclusiones

Dificultades encontradas

- Manejo de tipos de datos: al leer el CSV, todos los valores llegan como strings. Fue necesario convertir población y superficie a enteros con `int()` y agregar validaciones para detectar valores no numéricos.
- Ordenamiento por campo variable: para ordenar por diferentes campos (nombre, población, superficie) usando una misma función, se utilizó una función anidada que accede al parámetro campo de la función contenedora, aplicando el concepto de closure.
- Validación de duplicados: al agregar un país, se implementó una comparación case-insensitive para evitar duplicados con diferente capitalización.

Conclusiones

El desarrollo de este TPI permitió consolidar los conceptos fundamentales de Programación 1, integrando en un sistema real el uso de listas, diccionarios, funciones, condicionales, manejo de archivos y estructuras repetitivas.

La aplicación del principio de modularización resultó clave para mantener el código organizado y comprensible. Cada función con responsabilidad única facilita la detección de errores y la eventual extensión del sistema.

El trabajo con archivos CSV introdujo la necesidad de manejar errores en la lectura y conversión de datos, lo que reforzó el uso de `try/except` como herramienta de robustez.

5. Bibliografía / Webgrafía

- Python Software Foundation. (2024). Python 3 Documentation - Built-in Types: Lists. <https://docs.python.org/3/library/stdtypes.html#lists>
- Python Software Foundation. (2024). Python 3 Documentation - Built-in Types: Dictionaries. <https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>
- Python Software Foundation. (2024). Python 3 Documentation - csv module. <https://docs.python.org/3/library/csv.html>
- Python Software Foundation. (2024). Python 3 Documentation - os.path module. <https://docs.python.org/3/library/os.path.html>
- Python Software Foundation. (2024). Python 3 Documentation - sorted() built-in function. <https://docs.python.org/3/library/functions.html#sorted>
- Python Software Foundation. (2024). Python 3 Documentation - Defining Functions. <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>
- Real Python. (2024). Reading and Writing CSV Files in Python. <https://realpython.com/python-csv/>
- Python Software Foundation. (2024). Python 3 Documentation - Lambda Expressions. <https://docs.python.org/3/reference/expressions.html#lambda>

6. Links

Repositorio GitHub: <https://github.com/RodoGaspar/TPI-Gestion-datos-paises>

Video demostrativo: <https://youtu.be/OHZq47HXOGU>